ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE TÉTOUAN

3ème année Cycle d'ingénieurs – Génie Informatique

Projet Business Intelligence

GreenCity Energy Management

Conception et Mise en Œuvre d'un Système Décisionnel Complet

Système de Gestion Énergétique des Bâtiments Intelligents

Extraction, Transformation et Chargement (ETL) de données hétérogènes
Analyse de la Consommation Énergétique, Rentabilité et Impact Environnemental

Réalisé par :
BELOUARRAQ Mohammed
ELKHOUMSI Imane

Encadré par :
Prof. Yacine EL YOUNOUSSI

Année Universitaire 2025-2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de Business Intelligence.

Nos remerciements s'adressent en premier lieu à notre encadrant, le Professeur **Y. EL YOUNOUSSI**, pour ses conseils précieux, son soutien continu et son expertise tout au long de ce projet. Sa disponibilité et ses orientations nous ont été d'une aide inestimable.

Nous remercions également l'ensemble du corps professoral de l'**École Nationale des Sciences Appliquées de Tétouan** pour la qualité de la formation dispensée en Business Intelligence et en Génie Informatique.

Nous exprimons notre reconnaissance à l'administration de l'ENSA de Tétouan pour avoir mis à notre disposition les ressources nécessaires à la réalisation de ce projet.

Enfin, nous remercions nos familles et nos proches pour leur soutien moral et leurs encouragements tout au long de notre parcours académique.

L'équipe GreenCity

Résumé

Ce rapport présente la conception et la mise en œuvre d'un système décisionnel complet pour **GreenCity**, une entreprise fictive spécialisée dans la gestion énergétique des bâtiments intelligents. Le projet s'inscrit dans le cadre du module Business Intelligence de la 3ème année du cycle d'ingénieurs en Génie Informatique à l'ENSA de Tétouan.

Contexte : GreenCity gère plusieurs bâtiments intelligents répartis sur différentes régions et souhaite centraliser ses données pour surveiller la consommation énergétique, évaluer la rentabilité économique et mesurer l'impact environnemental.

Objectif : Concevoir un Data Warehouse composé de trois Data Marts thématiques indépendants (Consommation Énergétique, Rentabilité Économique, Impact Environnemental) alimentés par un processus ETL automatisé à partir de sources de données hétérogènes (MySQL, fichiers JSON et CSV).

Méthodologie : Le projet a suivi l'approche dimensionnelle de Kimball avec une architecture en étoile. Le processus ETL a été implémenté en utilisant **Pentaho Data Integration (PDI)** pour les transformations visuelles et **Python** pour les scripts automatisés. L'automatisation a été mise en place avec **Apache Airflow**. Le reporting a été réalisé avec **Power BI**.

Résultats : Le système développé permet l'analyse de plus de 22 000 enregistrements répartis sur 3 tables de faits et 6 dimensions. Plus de 30 KPI ont été implémentés et visualisés à travers des dashboards interactifs. Le processus ETL est entièrement automatisé et s'exécute quotidiennement avec extraction incrémentale.

Mots-clés : Business Intelligence, Data Warehouse, Data Mart, ETL, Pentaho, MySQL, Power BI, Architecture en Étoile, Kimball, Consommation Énergétique, Smart Buildings, Apache Airflow.

Table des matières

Remerciements **1**

Résumé **2**

Liste des abréviations **8**

Introduction Générale **9**

1 Contexte du Projet et État de l'Art **10**
1.1 Introduction . . . 10
1.2 Présentation de GreenCity . . . 10
1.2.1 Problématique . . . 10
1.2.2 Objectifs du projet . . . 11
1.3 Concepts de Business Intelligence . . . 11
1.3.1 Définition . . . 11
1.3.2 Architecture BI . . . 12
1.3.3 OLTP vs OLAP . . . 12
1.3.4 Modélisation dimensionnelle . . . 12
1.4 Processus ETL . . . 12
1.4.1 Trois phases . . . 12
1.5 Technologies utilisées . . . 12
1.6 Méthodologie adoptée . . . 13
1.7 Conclusion . . . 13

2 Analyse des Besoins et Spécification **14**
2.1 Introduction . . . 14
2.2 Trois axes d'analyse . . . 14
2.2.1 Axe 1 : Consommation Énergétique . . . 14
2.2.2 Axe 2 : Rentabilité Économique . . . 14
2.2.3 Axe 3 : Impact Environnemental . . . 14
2.3 Indicateurs Clés de Performance (KPI) . . . 14
2.3.1 Exemples de KPI principaux . . . 15
2.4 Sources de données . . . 15
2.5 Exigences principales . . . 15
2.5.1 Exigences fonctionnelles . . . 15
2.5.2 Exigences non fonctionnelles . . . 15
2.6 Conclusion . . . 15

3 Conception du Data Warehouse **16**
3.1 Introduction . . . 16

3.2 Architecture globale . . . 16
3.3 Data Mart 1 : Consommation Énergétique . . . 17
3.3.1 Caractéristiques . . . 17
3.4 Data Mart 2 : Rentabilité Économique . . . 18
3.4.1 Caractéristiques . . . 18
3.5 Data Mart 3 : Impact Environnemental . . . 18
3.5.1 Caractéristiques . . . 19
3.6 Dimensions conformes . . . 19
3.7 Gestion de l'historique : SCD Type 2 . . . 19
3.8 Vues analytiques . . . 19
3.9 Conclusion . . . 20

4 Processus ETL 21
4.1 Introduction . . . 21
4.2 Architecture du processus ETL . . . 21
4.2.1 Vue d'ensemble . . . 21
4.2.2 Zones de données . . . 21
4.2.3 Technologies ETL . . . 21
4.3 Phase 1 : Extraction . . . 22
4.3.1 Objectif . . . 22
4.3.2 Extraction depuis MySQL . . . 22
4.3.3 Extraction depuis fichiers JSON . . . 24
4.4 Phase 2 : Transformation . . . 26
4.4.1 Objectif . . . 26
4.4.2 Sous-phase 2.1 : Nettoyage . . . 26
4.4.3 Sous-phase 2.2 : Enrichissement . . . 27
4.4.4 Sous-phase 2.3 : Préparation des dimensions . . . 29
4.4.5 Sous-phase 2.4 : Préparation des faits . . . 30
4.5 Phase 3 : Chargement . . . 31
4.5.1 Objectif . . . 31
4.5.2 Chargement des dimensions . . . 31
4.5.3 Chargement des faits . . . 33
4.5.4 Gestion des contraintes d'intégrité . . . 34
4.6 Monitoring et gestion des erreurs . . . 34
4.6.1 Tables de log . . . 34
4.6.2 Gestion des erreurs . . . 35
4.6.3 Fichier de log . . . 36
4.7 Statistiques de réalisation . . . 36
4.7.1 Volumétrie traitée . . . 36
4.7.2 Performance . . . 37
4.8 Conclusion . . . 37

5 Automatisation et Reporting 38
5.1 Introduction . . . 38
5.2 Automatisation du processus ETL . . . 38
5.2.1 Besoins d'automatisation . . . 38
5.2.2 Planification des exécutions . . . 39
5.2.3 Monitoring et gestion des logs . . . 40
5.3 Reporting et Visualisation avec Power BI . . . 40

5.3.1 Architecture du reporting 40
5.4 Tableaux de bord décisionnels 41
5.5 Conclusion 42

6 Conclusion Générale **43**
6.1 Synthèse du projet 43
6.1.1 Réalisations principales 43
6.2 Bénéfices pour GreenCity 44
6.3 Compétences acquises 44
6.3.1 Compétences techniques 44
6.3.2 Compétences méthodologiques 45
6.4 Conclusion finale 45

Bibliographie **46**

Table des figures

1.1 Le problématique 11
1.2 Objectifs du projet 11

3.1 Data Mart 1 : Consommation Énergétique 17
3.2 Data Mart 2 : Rentabilité Économique 18
3.3 Data Mart 3 : Impact Environnemental 18

4.1 Les 6 phases du transformation des données 21
4.2 Transformation Pentaho - Extraction MySQL 23
4.3 Logs d'exécution Pentaho - Extraction 24
4.4 Transformation Pentaho - Extraction JSON 25
4.5 Transformation Pentaho - Nettoyage Clients 26
4.6 Transformation Pentaho - Enrichissement Factures 28
4.7 Transformation Pentaho - Préparation $\text{dim}_{c}lient$ 29
4.8 Transformation Pentaho - Préparation $\text{fact}_{c}onsommation$ 30
4.9 Transformation Pentaho - Préparation $\text{fact}_{e}nvironnement$ 31
4.10 Transformation Pentaho - Chargement Dimensions 32
4.11 Transformation Pentaho - Chargement Faits 33

5.1 Job Pentaho ETL - Orchestration complète 39
5.2 Suivi de la consommation énergétique 41
5.3 Suivi de la rentabilité économique 42
5.4 Suivi de l'impact environnemental 42

Liste des tableaux

2.1 Principaux KPI par axe d'analyse 15
2.2 Sources de Données Opérationnelles 15

3.1 Dimensions conformes du Data Warehouse 19

4.1 Problèmes de qualité des données 26
4.2 Volumétrie traitée par l'ETL 36

Liste des abréviations

BI	Business Intelligence (Intelligence d'Affaires)
CA	Chiffre d'Affaires
CSV	Comma-Separated Values (Valeurs séparées par des virgules)
DW	Data Warehouse (Entrepôt de Données)
ETL	Extract, Transform, Load (Extraire, Transformer, Charger)
ENSA	École Nationale des Sciences Appliquées
JSON	JavaScript Object Notation
KPI	Key Performance Indicator (Indicateur Clé de Performance)
OLAP	Online Analytical Processing (Traitement Analytique en Ligne)
OLTP	Online Transaction Processing (Traitement Transactionnel en Ligne)
PDI	Pentaho Data Integration
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language (Langage de Requête Structuré)
SCD	Slowly Changing Dimension (Dimension à Évolution Lente)
TTC	Toutes Taxes Comprises
HT	Hors Taxes
TVA	Taxe sur la Valeur Ajoutée
IoT	Internet of Things (Internet des Objets)
CO_2	Dioxyde de Carbone
kWh	Kilowatt-heure
m^3	Mètre cube
MAD	Dirham Marocain

Introduction Générale

À l'ère de la transformation numérique, les organisations produisent et collectent quotidiennement des volumes considérables de données issues de sources multiples et hétérogènes. Cette explosion des données représente à la fois un défi majeur — lié à leur gestion, leur intégration et leur qualité — et une opportunité stratégique, à condition de pouvoir les exploiter efficacement pour soutenir la prise de décision.

Dans ce contexte, la **Business Intelligence (BI)** s'impose comme une approche technologique et méthodologique essentielle permettant de transformer des données brutes en informations pertinentes et exploitables. Les systèmes décisionnels modernes reposent sur des architectures structurées intégrant des Data Warehouses, des processus ETL (Extract, Transform, Load) robustes et des outils de visualisation interactifs destinés aux décideurs.

GreenCity, entreprise spécialisée dans la gestion énergétique des bâtiments intelligents, illustre parfaitement cette problématique. Elle administre un ensemble de bâtiments répartis sur plusieurs régions et collecte des données provenant de diverses sources :

- des capteurs IoT mesurant la consommation d'électricité, d'eau et de gaz,
- un système transactionnel MySQL gérant les clients, les factures et les paiements,
- des rapports environnementaux sous forme de fichiers CSV (émissions de CO_2, taux de recyclage).

Ces données, dispersées et de formats variés, ne peuvent être exploitées efficacement sans un processus de centralisation, de nettoyage et de structuration. GreenCity exprime ainsi le besoin de disposer d'une vision globale et fiable afin de suivre sa consommation énergétique, d'évaluer sa rentabilité économique et de mesurer son impact environnemental à l'aide d'indicateurs clés de performance (KPI).

L'objectif principal de ce projet est de concevoir et de mettre en œuvre un système décisionnel complet répondant à ces besoins. Ce système repose sur la création de trois Data Marts thématiques indépendants — *Consommation Énergétique*, *Rentabilité Économique* et *Impact Environnemental* — alimentés par un processus ETL automatisé et exploités à travers des tableaux de bord interactifs développés avec Power BI.

Le projet s'appuie sur l'approche dimensionnelle de Kimball avec une modélisation en étoile, utilise **Pentaho Data Integration** et **Python** pour l'implémentation des processus ETL, et adopte **MySQL** comme système de gestion du Data Warehouse.

Chapitre 1

Contexte du Projet et État de l'Art

1.1 Introduction

Ce chapitre présente le contexte du projet GreenCity, les concepts fondamentaux de la Business Intelligence et les technologies utilisées.

1.2 Présentation de GreenCity

GreenCity est une entreprise fictive spécialisée dans la gestion énergétique des bâtiments intelligents. Elle gère 50 bâtiments répartis sur 5 régions, équipés de 500 compteurs intelligents collectant des mesures horaires de consommation (électricité, eau, gaz).

1.2.1 Problématique

GreenCity collecte des données à partir de trois sources hétérogènes :

— **Systèmes IoT** : fichiers JSON (consommation horaire)

— **Base MySQL** : clients, factures, paiements, bâtiments

— **Rapports CSV** : émissions CO_2, taux de recyclage

Ces données sont dispersées, de qualité variable (doublons, valeurs manquantes, formats incohérents) et difficiles à analyser de manière transversale.



FIGURE 1.1 – Le problématique

1.2.2 Objectifs du projet

GreenCity souhaite :

— **Centraliser** les données dans un Data Warehouse

— **Analyser** trois axes décisionnels : Consommation Énergétique, Rentabilité Économique, Impact Environnemental

— **Automatiser** le processus ETL

— **Visualiser** les KPI via des dashboards interactifs

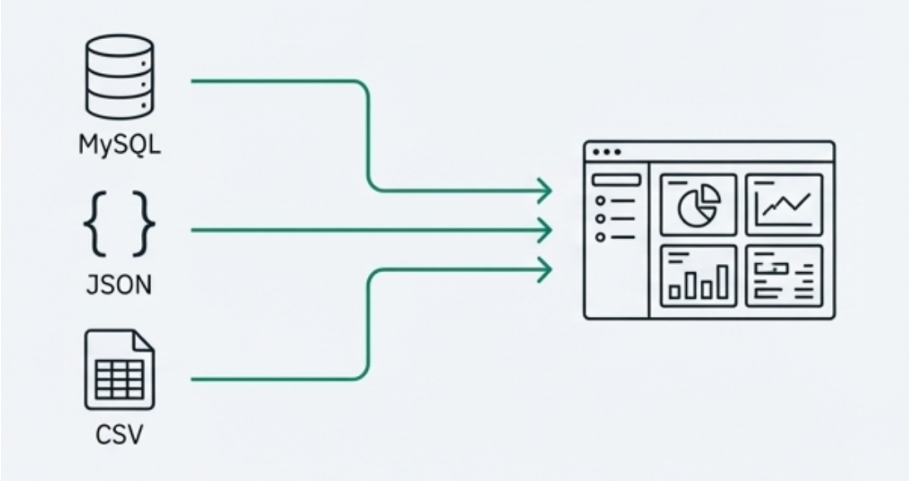


FIGURE 1.2 – Objectifs du projet

1.3 Concepts de Business Intelligence

1.3.1 Définition

La **Business Intelligence (BI)** transforme les données brutes en informations exploitables pour améliorer la prise de décision. Elle repose sur quatre piliers : intégration des données, qualité des données, stockage analytique (Data Warehouse) et visualisation.

1.3.2 Architecture BI

L'architecture comprend :

— **Sources** : MySQL, JSON, CSV
— **Staging** : zone temporaire pour les données extraites
— **ETL** : Extraction, Transformation, Chargement
— **Data Warehouse** : stockage analytique optimisé
— **Reporting** : dashboards Power BI

1.3.3 OLTP vs OLAP

— **OLTP (Opérationnel)** : transactions courantes, données actuelles, modèle normalisé
— **OLAP (Analytique)** : analyse décisionnelle, données historiques, modèle dimensionnel (étoile)

1.3.4 Modélisation dimensionnelle

La modélisation dimensionnelle (approche Kimball) repose sur :

— **Tables de faits** : mesures numériques (consommation, CA, émissions CO_2)
— **Dimensions** : contexte d'analyse (date, région, bâtiment, client, type d'énergie)
— **Schéma en étoile** : table de faits au centre, dimensions autour
— **SCD Type 2** : historisation des changements dans les dimensions

Avantages : simplicité, performance, flexibilité.

1.4 Processus ETL

L'ETL (*Extract, Transform, Load*) alimente le Data Warehouse à partir des sources opérationnelles.

1.4.1 Trois phases

— **Extraction (E)** : récupération des données depuis MySQL, JSON, CSV (extraction incrémentale)
— **Transformation (T)** : nettoyage (doublons, valeurs manquantes), enrichissement (calculs, jointures), conformité
— **Chargement (L)** : insertion dans le DW avec gestion SCD Type 2 pour les dimensions

1.5 Technologies utilisées

— **Pentaho Data Integration (PDI)** : transformations ETL visuelles (.ktr) et jobs (.kjb)

— **Python** : scripts d'automatisation (pandas, mysql-connector, json)
— **MySQL 8.0** : stockage opérationnel et Data Warehouse
— **Power BI** : dashboards interactifs avec langage DAX
— **Cron/Task Scheduler** : planification des exécutions

1.6 Méthodologie adoptée

Ce projet adopte l'approche Kimball (Bottom-Up) avec construction par Data Marts thématiques, modélisation dimensionnelle en étoile et dimensions conformes.

1.7 Conclusion

Ce chapitre a présenté le contexte de GreenCity, les concepts fondamentaux de la BI (Data Warehouse, modélisation dimensionnelle, ETL) et les technologies retenues (Pentaho, Python, MySQL, Power BI).

Le prochain chapitre détaille l'analyse des besoins et la spécification des trois Data Marts.

Chapitre 2

Analyse des Besoins et Spécification

2.1 Introduction

Ce chapitre présente les trois axes d'analyse prioritaires, les KPI essentiels, les sources de données et les exigences du système.

2.2 Trois axes d'analyse

2.2.1 Axe 1 : Consommation Énergétique

Objectif : Analyser la consommation (électricité, eau, gaz) par compteur, bâtiment, région et période.

Grain : une mesure par compteur et par heure.

Questions clés : Quelle est la consommation totale ? Quels sont les bâtiments les plus consommateurs ? Corrélation température-consommation ?

2.2.2 Axe 2 : Rentabilité Économique

Objectif : Analyser le CA, les marges, le recouvrement et les délais de paiement.

Grain : une facture (avec paiement éventuel).

Questions clés : Quel est le CA par région ? Quels clients sont rentables ? Quel est le taux de recouvrement ?

2.2.3 Axe 3 : Impact Environnemental

Objectif : Mesurer les émissions CO_2 et le taux de recyclage.

Grain : un rapport mensuel par bâtiment.

Questions clés : Quelles sont les émissions totales ? Quels bâtiments polluent le plus ? Sommes-nous conformes aux normes ?

2.3 Indicateurs Clés de Performance (KPI)

Plus de 30 KPI ont été définis pour les trois axes d'analyse.

2.3.1 Exemples de KPI principaux

TABLE 2.1 – Principaux KPI par axe d'analyse

Axe	KPI	Unité
Consommation	Consommation totale	kWh
	Top 10 bâtiments consommateurs	kWh
	Consommation par m^2	kWh/m^2
Rentabilité	Chiffre d'affaires total	MAD
	Taux de marge moyen	%
	Taux de recouvrement	%
Environnement	Émissions CO_2 totales	kg
	Taux de recyclage moyen	%
	Ratio CO_2/consommation	kg/kWh

2.4 Sources de données

TABLE 2.2 – Sources de Données Opérationnelles

Source	Format	Volumétrie
Système IoT	JSON (36 fichiers/an)	16 000 mesures/jour
Facturation	MySQL (9 tables)	5 000 factures/an
Environnement	CSV (12 fichiers/an)	600 rapports/an

2.5 Exigences principales

2.5.1 Exigences fonctionnelles

— **ETL** : extraction incrémentale, nettoyage (doublons, valeurs manquantes), enrichissement, chargement avec SCD Type 2

— **Reporting** : dashboards interactifs avec filtres, drill-down, export PDF/Excel

2.5.2 Exigences non fonctionnelles

— **Performance** : ETL $<$ 1 heure, requêtes $<$ 5 secondes

— **Fiabilité** : taux de succès $>$ 99%, reprise automatique

— **Scalabilité** : support 100 bâtiments, 5 ans d'historique

2.6 Conclusion

Ce chapitre a identifié les trois axes d'analyse (Consommation, Rentabilité, Environnement), défini les principaux KPI, spécifié les sources de données et formulé les exigences du système.

Le prochain chapitre détaille la conception du Data Warehouse et la modélisation dimensionnelle.

Chapitre 3

Conception du Data Warehouse

3.1 Introduction

Ce chapitre présente l'architecture du Data Warehouse GreenCity et la modélisation des trois Data Marts.

3.2 Architecture globale

Le Data Warehouse adopte une **architecture en constellation** (*fact constellation*) composée de :

- **3 tables de faits** : fact_consommation, fact_rentabilite, fact_environnement
- **6 dimensions conformes** : dim_date, dim_region, dim_batiment, dim_client, dim_type_energie, dim_compteur
- **Historisation** : SCD Type 2 pour les dimensions
- **Volumétrie** : 22 000 enregistrements de faits

3.3 Data Mart 1 : Consommation Énergétique

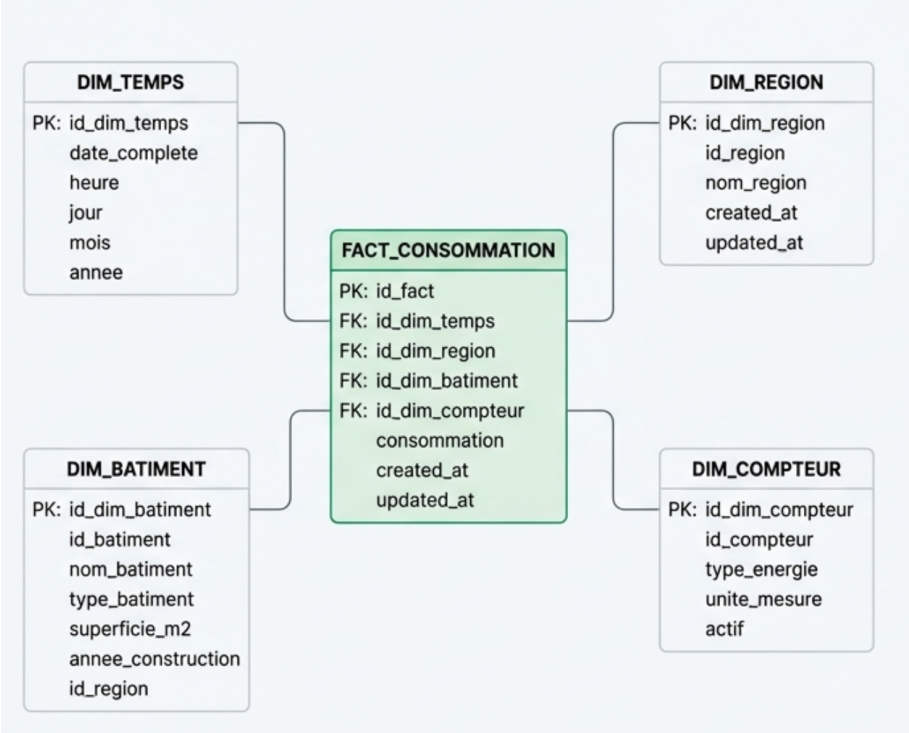


FIGURE 3.1 – Data Mart 1 : Consommation Énergétique

3.3.1 Caractéristiques

- **Objectif** : analyser la consommation (électricité, eau, gaz) par compteur, bâtiment, région et période
- **Grain** : une mesure par compteur et par heure
- **Dimensions** : date, compteur, bâtiment, région, client, type_energie
- **Mesures** : consommation_kwh, consommation_m3, consommation_normalisee, cout_total, temperature, consommation_par_m2
- **Volumétrie** : 12 000 lignes/jour, 4 380 000 lignes/an

3.4 Data Mart 2 : Rentabilité Économique

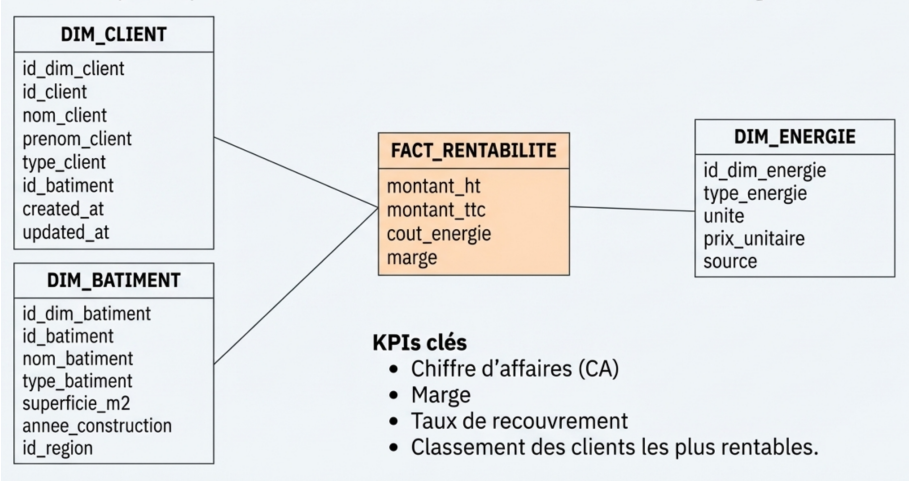


FIGURE 3.2 – Data Mart 2 : Rentabilité Économique

3.4.1 Caractéristiques

— **Objectif** : analyser le CA, les marges, le recouvrement et les délais de paiement

— **Grain** : une facture (avec paiement éventuel)

— **Dimensions** : date_facture, date_paiement, date_echeance, client, bâtiment, région, type_energie

— **Mesures** : montant_ttc, marge, taux_marge, delai_paiement, retard_paiement, est_paye

— **Particularité** : dimension date à rôles multiples (*role-playing dimension*)

— **Volumétrie** : 5 100 lignes/an

3.5 Data Mart 3 : Impact Environnemental

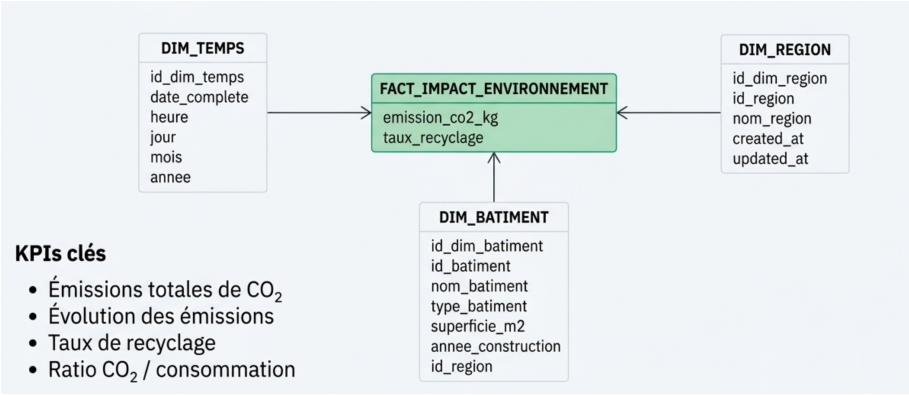


FIGURE 3.3 – Data Mart 3 : Impact Environnemental

3.5.1 Caractéristiques

— **Objectif** : analyser les émissions CO_2, le taux de recyclage et la conformité environnementale

— **Grain** : un rapport mensuel par bâtiment

— **Type** : snapshot périodique mensuel

— **Dimensions** : date, bâtiment, région

— **Mesures** : emission_co2_kg, taux_recyclage, ratio_co2_consommation, emission_co2_par_m2 est_conforme_norme

— **Volumétrie** : 600 lignes/an

3.6 Dimensions conformes

Les dimensions conformes sont partagées par les trois Data Marts, garantissant la cohérence sémantique.

TABLE 3.1 – Dimensions conformes du Data Warehouse

Dimension	Attributs principaux	SCD
dim_date	jour, mois, annee, trimestre, nom_mois, est_weekend	–
dim_region	code_region, nom_region, zone_climatique, population	Type 2
dim_batiment	code_batiment, nom, type, surface_m2, certification	Type 2
dim_client	code_client, type_client, nom, segment	Type 2
dim_compteur	code_compteur, numero_serie, marque, date_installation	Type 2
dim_type_energie	nom_energie (Électricité, Eau, Gaz), unite, facteur_co2	Type 1

3.7 Gestion de l'historique : SCD Type 2

Le SCD Type 2 historise les changements en créant une nouvelle ligne avec dates de validité (date_debut, date_fin, est_actuel).

Avantages : historique complet, analyses temporelles, traçabilité.

Exemple : un bâtiment change de certification énergétique (D → C → B) : 3 lignes avec le même code_batiment mais différentes périodes de validité.

3.8 Vues analytiques

Des vues SQL ont été créées pour faciliter les analyses courantes :

— **v_consommation_region_mois** : agrégation de la consommation par région, mois et type d'énergie

— **v_rentabilite_client** : calcul du CA, marge et taux de recouvrement par client
— **v_environnement_batiment** : synthèse des performances environnementales par bâtiment

3.9 Conclusion

Ce chapitre a présenté l'architecture du Data Warehouse en constellation de faits, composée de trois Data Marts (Consommation, Rentabilité, Environnement) partageant six dimensions conformes. L'utilisation du SCD Type 2 assure l'historisation des changements.

Le prochain chapitre détaille le processus ETL mis en place pour alimenter ce Data Warehouse.

Chapitre 4

Processus ETL

4.1 Introduction

Ce chapitre présente le processus ETL (*Extract, Transform, Load*) développé pour alimenter le Data Warehouse GreenCity. Nous décrivons les trois phases (Extraction, Transformation, Chargement), les transformations Pentaho, les scripts Python, ainsi que la gestion de la qualité des données et la traçabilité.

4.2 Architecture du processus ETL

4.2.1 Vue d'ensemble

Le processus ETL suit une architecture en trois phases, intégrant une zone de staging pour isoler l'extraction des traitements et du chargement.

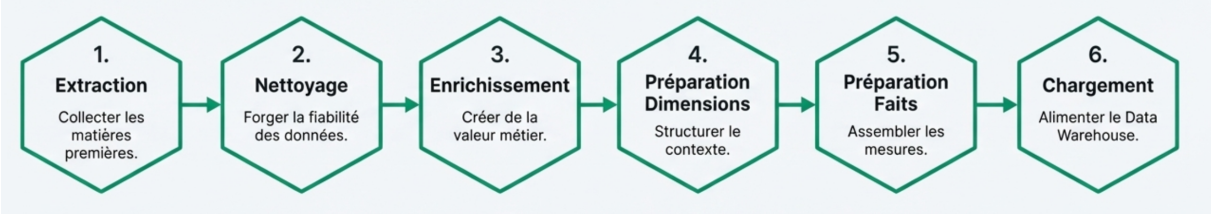


FIGURE 4.1 – Les 6 phases du transformation des données

4.2.2 Zones de données

- **Sources** : MySQL, JSON (36 fichiers), CSV (12 fichiers) ;
- **Staging – Extracted** : fichiers CSV extraits (données brutes extraites) ;
- **Staging – Transformed** : fichiers CSV transformés (nettoyés et enrichis) ;
- **Data Warehouse** : 3 tables de faits et 6 dimensions.

4.2.3 Technologies ETL

- **Pentaho Data Integration (PDI)** : transformations (`.ktr`) et job (`.kjb`) ;
- **Python 3** : scripts d'automatisation et de consolidation ;
- **MySQL** : stockage opérationnel (sources) et stockage analytique (DW).

4.3 Phase 1 : Extraction

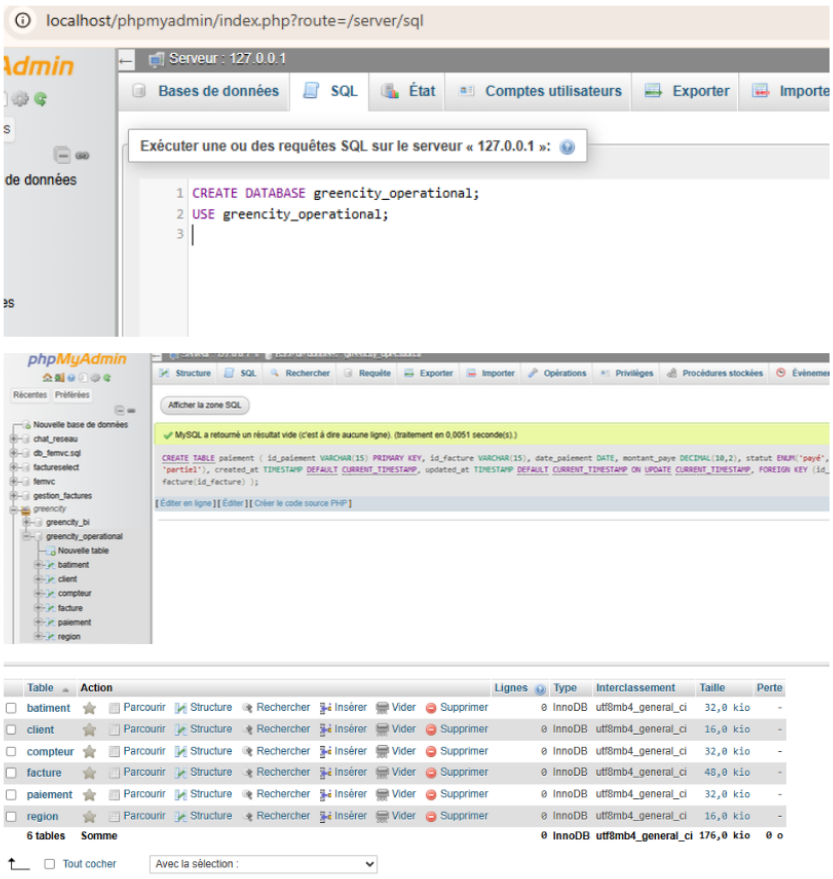
4.3.1 Objectif

Récupérer les données depuis les sources opérationnelles et les déposer dans la zone de staging afin de garantir une séparation claire entre l'extraction et les traitements.

4.3.2 Extraction depuis MySQL

Base de données opérationnelle

La Figure présente la structure de la base de données opérationnelle MySQL.



Tables extraites

- `dim_regions` → `staging/extracted/regions.csv`
- `dim_clients` → `staging/extracted/clients.csv`
- `dim_batiments` → `staging/extracted/batiments.csv`
- `dim_compteurs` → `staging/extracted/compteurs.csv`
- `fact_factures` + `fact_paiements` → `staging/extracted/factures.csv`

Extraction incrémentale

L'extraction est **incrémentale** et s'appuie sur une colonne de suivi (ex. `date_modification`) ainsi que sur une table de log d'extraction. Le Listing ci-dessous illustre le principe.

```sql
SELECT *
FROM dim_clients
WHERE date_modification > (
    SELECT date_derniere_extraction
    FROM log_extraction
    WHERE nom_table = 'dim_clients'
    ORDER BY date_derniere_extraction DESC
    LIMIT 1
);
```

Listing 4.1 – Exemple d'extraction incrémentale MySQL

Transformation Pentaho : Extraction MySQL

La Figure montre la transformation Pentaho utilisée pour extraire les données depuis MySQL.



FIGURE 4.2 – Transformation Pentaho - Extraction MySQL

Étapes principales :

1. **Table Input** : connexion MySQL et requête d'extraction ;
2. **Get System Info** : ajout du timestamp d'extraction ;
3. **Select Values** : sélection, renommage et typage des colonnes ;
4. **Text File Output** : écriture dans un fichier CSV ;
5. **Insert/Update** : mise à jour de la table `log_extraction`.

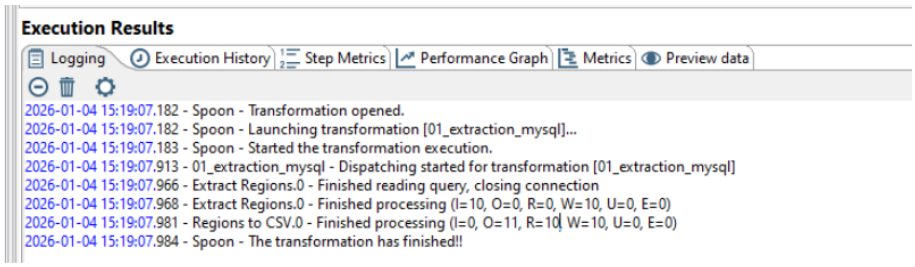


FIGURE 4.3 – Logs d'exécution Pentaho - Extraction

4.3.3 Extraction depuis fichiers JSON

Fichiers traités

Les données de consommation sont fournies sous forme de 36 fichiers JSON (3 types × 12 mois) :

- `Elec_consumption_01_2024.json` ...`Elec_consumption_12_2024.json`
- `Eau_consumption_01_2024.json` ...`Eau_consumption_12_2024.json`
- `Gaz_consumption_01_2024.json` ...`Gaz_consumption_12_2024.json`

Parsing JSON (Python)

Le script suivant illustre le principe d'extraction JSON vers CSV. Il convertit le format imbriqué en un format tabulaire exploitable.

```python
import json
import pandas as pd

def extract_json_to_csv(json_file, output_csv):
    with open(json_file, "r", encoding="utf-8") as f:
        data = json.load(f)

    rows = []
    for batiment in data.get("batiments", []):
        for mesure in batiment.get("mesures", []):
            rows.append({
                "id_region": data.get("id_region"),
                "id_batiment": batiment.get("id_batiment"),
                "compteur_id": mesure.get("compteur_id"),
                "date_mesure": mesure.get("date_mesure"),
                "consommation": (
                    mesure.get("consommation_kWh")
                    if mesure.get("consommation_kWh") is not None
                    else mesure.get("consommation_m3")
                ),
                "unite": batiment.get("unite"),
                "type_energie": batiment.get("type_energie"),
                "temperature_celsius": mesure.get("
                   temperature_celsius"),
```

```
                "humidite_pourcent": mesure.get("humidite_pourcent"
                    ),
            })

    df = pd.DataFrame(rows)
    df.to_csv(output_csv, index=False)
    print(f"Extracted {len(df)} rows to {output_csv}")
```

Listing 4.2 – Extraction JSON vers CSV (principe)

Transformation Pentaho : Extraction JSON

Étapes :

1. **Get Files Names** : liste des fichiers JSON ;
2. **JSON Input** : parsing du JSON ;
3. **Split Fields** : extraction des champs imbriqués ;
4. **Text File Output** : écriture dans un CSV.



FIGURE 4.4 – Transformation Pentaho - Extraction JSON

Consolidation

Les 12 fichiers sont consolidés en un seul fichier : `staging/extracted/environnement.csv`.

```
import pandas as pd
from pathlib import Path

def consolidate_env_reports(input_dir, output_file):
    csv_files = sorted(Path(input_dir).glob("env_reports_*.csv"))

    dfs = []
    for csv_file in csv_files:
        dfs.append(pd.read_csv(csv_file))

    consolidated_df = pd.concat(dfs, ignore_index=True)
    consolidated_df.to_csv(output_file, index=False)
    print(f"Consolidated {len(consolidated_df)} rows from {len(
        csv_files)} files")
```

Listing 4.3 – Consolidation des CSV environnementaux

4.4 Phase 2 : Transformation

4.4.1 Objectif

Nettoyer, valider, enrichir et mettre en conformité les données extraites afin de produire des fichiers prêts pour le chargement dans le Data Warehouse.

4.4.2 Sous-phase 2.1 : Nettoyage

Problèmes de qualité détectés

Le Tableau synthétise les principaux problèmes et leurs solutions.

TABLE 4.1 – Problèmes de qualité des données

Problème	Exemple	Solution
Doublons	Même facture en double	Déduplication par clés métier
Valeurs manquantes	`email = NULL`	Valeur par défaut / exclusion
Espaces inutiles	`" Ali "`	`TRIM()` + normalisation
Casse incohérente	`"TaNGER", "tanger"`	`UPPER()/LOWER()`
Dates invalides	`"32/13/2024"`	Validation + standardisation
Valeurs aberrantes	`consommation = -50`	Exclusion / correction

Transformation Pentaho : Nettoyage Clients

La Figure présente le flux de nettoyage des clients.

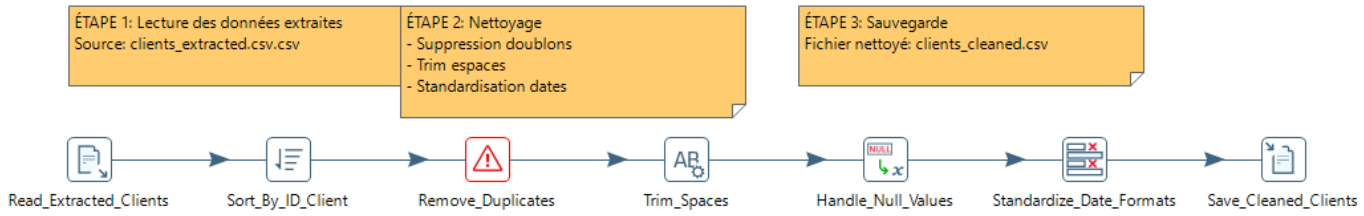


FIGURE 4.5 – Transformation Pentaho - Nettoyage Clients

Étapes de nettoyage :

1. **CSV File Input** : lecture du fichier extrait ;
2. **Unique Rows** : suppression des doublons ;
3. **Replace in String** : suppression des espaces multiples ;
4. **Trim Fields** : suppression des espaces avant/après ;
5. **Value Mapper** : normalisation de la casse ;
6. **Select Values** : conversion de types ;
7. **Filter Rows** : exclusion des valeurs invalides ;
8. **Text File Output** : génération du fichier nettoyé.

Script Python : nettoyage générique

```python
import pandas as pd
import numpy as np

def clean_dataframe(df):
    # Suppression des doublons
    df = df.drop_duplicates()

    # Nettoyage des champs texte
    for col in df.select_dtypes(include=["object"]).columns:
        df[col] = df[col].astype(str).str.strip()
        df[col] = df[col].str.replace(r"\s+", " ", regex=True)

    # Valeurs manquantes
    df = df.replace("", np.nan)

    # Standardisation des dates (si pr sentes)
    date_columns = ["date_facture", "date_paiement", "date_mesure"]
    for col in date_columns:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors="coerce")
            df[col] = df[col].dt.strftime("%Y-%m-%d")

    # Validation des valeurs num riques : n gatifs -> NaN
    for col in df.select_dtypes(include=["float64", "int64"]).
       columns:
        df.loc[df[col] < 0, col] = np.nan

    return df
```

Listing 4.4 – Fonction de nettoyage générique (principe)

4.4.3 Sous-phase 2.2 : Enrichissement

Objectif

Calculer des mesures dérivées, effectuer des jointures entre sources et produire des attributs calculés nécessaires aux KPI.

Transformation Pentaho : Enrichissement Factures

La Figure illustre l'enrichissement des factures.

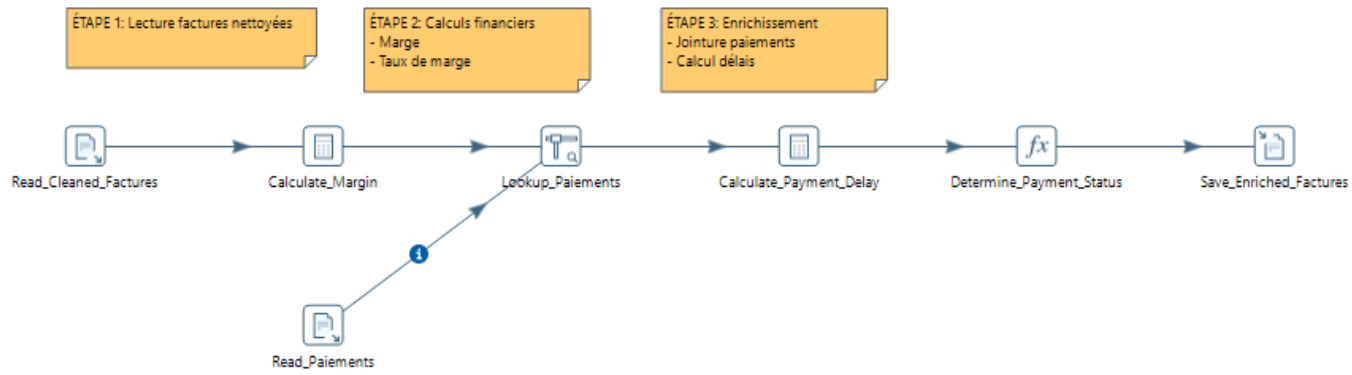


FIGURE 4.6 – Transformation Pentaho - Enrichissement Factures

Enrichissements réalisés :

1. **Marge** : `marge = montant_ttc - cout_energie` ;
2. **Taux de marge** : `taux_marge = marge / montant_ttc` ;
3. **Délai de paiement** : `DATEDIFF(date_paiement, date_facture)` ;
4. **Retard** : `DATEDIFF(date_paiement, date_echeance)` ;
5. **Indicateur est_paye** : paiement non NULL ;
6. **Indicateur est_en_retard** : retard > 0.

Script Python : enrichissement Consommation

```python
def enrich_consommation(df, df_compteurs, df_batiments,
    df_types_energie):
    # Jointure avec compteurs
    df = df.merge(df_compteurs, left_on="compteur_id", right_on="
        id_compteur", how="left")

    # Jointure avec b timents
    df = df.merge(df_batiments, on="id_batiment", how="left")

    # Jointure avec types d' nergie
    df = df.merge(df_types_energie, on="id_type_energie", how="left
        ")

    # Normalisation en kWh   quivalent   (exemple de facteurs)
    def normalize_to_kwh(row):
        if row["unite"] == "kWh":
            return row["consommation"]
        if row["unite"] == "m3" and str(row["type_energie"]).lower
            () == "eau":
            return row["consommation"] * 0.01
        if row["unite"] == "m3" and str(row["type_energie"]).lower
            () == "gaz":
            return row["consommation"] * 10.0
        return row["consommation"]
```

```
20
21      df["consommation_normalisee"] = df.apply(normalize_to_kwh, axis
            =1)
22
23      # Consommation par m2
24      df["consommation_par_m2"] = df["consommation_normalisee"] / df[
            "surface_m2"]
25
26      # Co ts (exemple)
27      df["cout_unitaire"] = df["type_energie"].map({
28          "   lectricit   ": 1.25,
29          "Eau": 8.50,
30          "Gaz": 0.85
31      })
32      df["cout_total"] = df["consommation"] * df["cout_unitaire"]
33
34      return df
```

Listing 4.5 – Enrichissement des données de consommation (principe)

4.4.4 Sous-phase 2.3 : Préparation des dimensions

Objectif

Transformer les données sources en un format conforme aux dimensions du Data Warehouse, notamment en ajoutant les attributs et colonnes nécessaires au SCD Type 2.

Transformation Pentaho : préparation de `dim_client`

La Figure illustre la préparation de la dimension client.

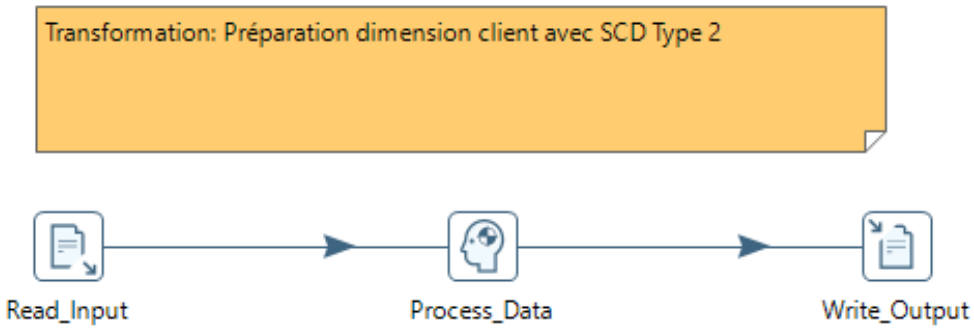


FIGURE 4.7 – Transformation Pentaho - Préparation dim$_{c}$*lient*

Étapes :

1. lecture du fichier `clients_cleaned.csv` ;
2. génération de l'attribut `segment_client` selon les règles métier ;
3. ajout des colonnes SCD Type 2 : `date_debut`, `date_fin`, `est_actuel` ;
4. génération d'un `code_client` unique (clé métier) ;
5. écriture dans `dim_client_prepared.csv`.

Dimensions préparées

- `dim_region_prepared.csv`
- `dim_batiment_prepared.csv`
- `dim_client_prepared.csv`
- `dim_compteur_prepared.csv`
- `dim_type_energie_prepared.csv` (statique)

4.4.5 Sous-phase 2.4 : Préparation des faits

Transformation Pentaho : préparation de `fact_consommation`

La Figure montre la préparation de la table de faits consommation.

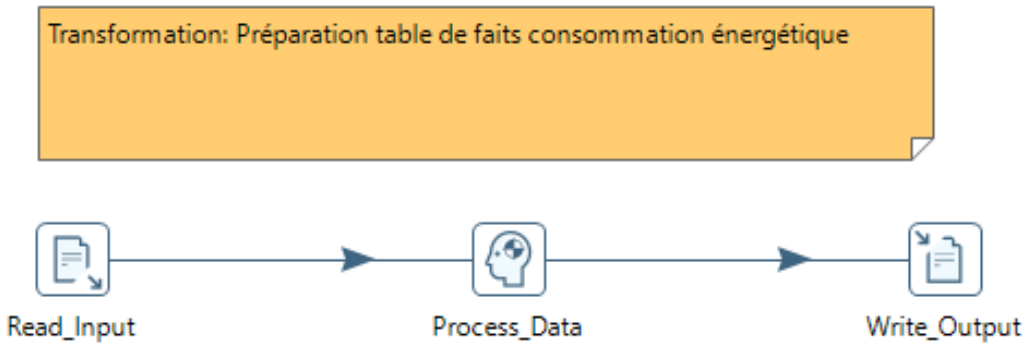


FIGURE 4.8 – Transformation Pentaho - Préparation fact$_c$*onsommation*

Étapes :

1. lecture des fichiers enrichis (électricité, eau, gaz) ;
2. extraction de l'heure à partir de `date_mesure` ;
3. calcul de `id_date` au format `YYYYMMDD` ;
4. *lookup* des clés étrangères (`id_batiment`, `id_region`, `id_client`, etc.) ;
5. sélection des colonnes finales ;
6. écriture dans `fact_consommation_prepared.csv`.

Transformation Pentaho : préparation de `fact_environnement`

La Figure illustre la préparation du fait environnemental.

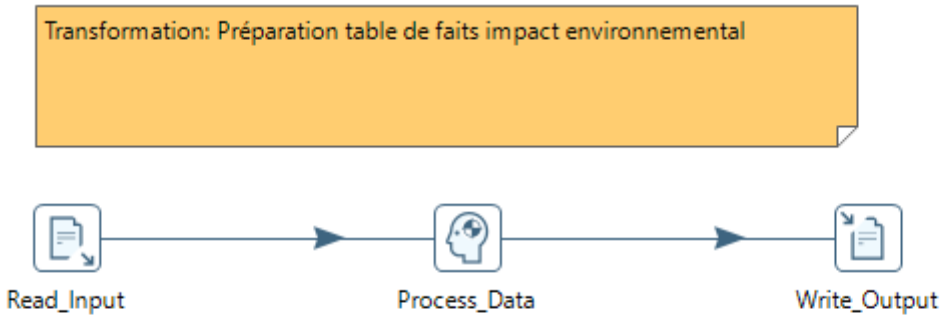


FIGURE 4.9 – Transformation Pentaho - Préparation fact$_e$*nvironnement*

Enrichissements spécifiques :

— calcul du `ratio_co2_consommation` ;
— calcul de `emission_co2_par_m2` ;
— comparaison avec le mois précédent ;
— calcul de `variation_emission_pourcent` ;
— détermination du `niveau_performance` (Excellent, Bon, Moyen, Faible) ;
— évaluation de la `conformité aux normes`.

4.5 Phase 3 : Chargement

4.5.1 Objectif

Charger les données transformées dans le Data Warehouse en respectant l'intégrité référentielle, l'historisation (SCD) et la traçabilité.

4.5.2 Chargement des dimensions

Stratégie : SCD Type 2

Les dimensions utilisent le SCD Type 2 afin d'historiser les changements. Le Listing ci-dessous illustre le principe général (exemple pédagogique).

```
def load_dimension_scd_type2(cursor, table_name, new_records):
    for record in new_records:
        business_key = record["code"]

        cursor.execute(f"""
            SELECT * FROM {table_name}
```

```
            WHERE code = %s AND est_actuel = TRUE
        """, (business_key,))

        existing = cursor.fetchone()

        if existing is None:
            cursor.execute(f"INSERT INTO {table_name} (...) VALUES
               (...)", tuple(record.values()))
        else:
            if has_changed(existing, record):
                cursor.execute(f"""
                    UPDATE {table_name}
                    SET date_fin = CURDATE(), est_actuel = FALSE
                    WHERE id = %s
                """, (existing["id"],))

                cursor.execute(f"INSERT INTO {table_name} (...)
                   VALUES (...)", tuple(record.values()))
```

Listing 4.6 – Principe de chargement SCD Type 2 (exemple)

Transformation Pentaho : chargement des dimensions

La Figure présente le chargement des dimensions.

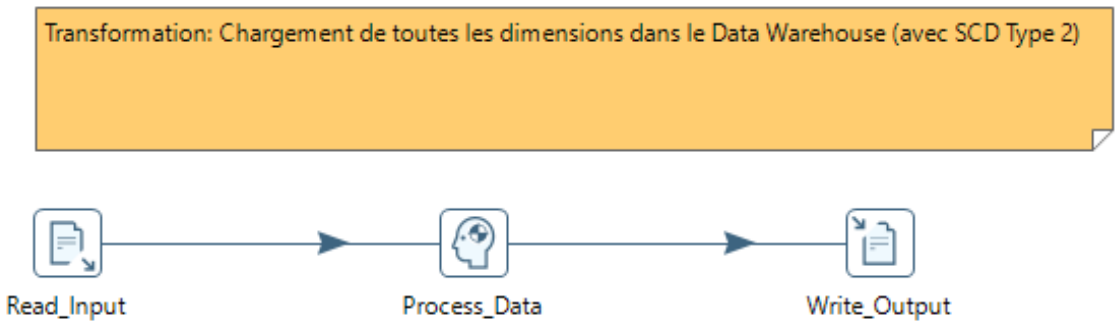


FIGURE 4.10 – Transformation Pentaho - Chargement Dimensions

Étapes :

1. **CSV File Input** : lecture des dimensions préparées ;
2. **Dimension Lookup/Update** : gestion SCD Type 2 ;
3. **Database Connection** : connexion au Data Warehouse ;
4. **Insert/Update** : chargement dans MySQL.

4.5.3 Chargement des faits

Stratégie : chargement incrémental

Les faits sont chargés de manière incrémentale. Le Listing ci-dessous illustre le principe de filtrage par dernière date de chargement (exemple pédagogique).

```python
import pandas as pd

def load_fact_incremental(cursor, fact_table, prepared_csv):
    df = pd.read_csv(prepared_csv)

    cursor.execute(f"SELECT MAX(date_ajout) AS last_load FROM {fact_table}")
    last_load = cursor.fetchone()["last_load"]

    if last_load is not None and "date_mesure" in df.columns:
        df = df[df["date_mesure"] > str(last_load)]

    if len(df) > 0:
        values = [tuple(row) for row in df.values]
        cursor.executemany(f"INSERT INTO {fact_table} (...) VALUES (...)", values)
        print(f"Loaded {len(df)} new records into {fact_table}")
```

Listing 4.7 – Principe de chargement incrémental des faits (exemple)

Transformation Pentaho : chargement des faits

La Figure présente le chargement des faits.

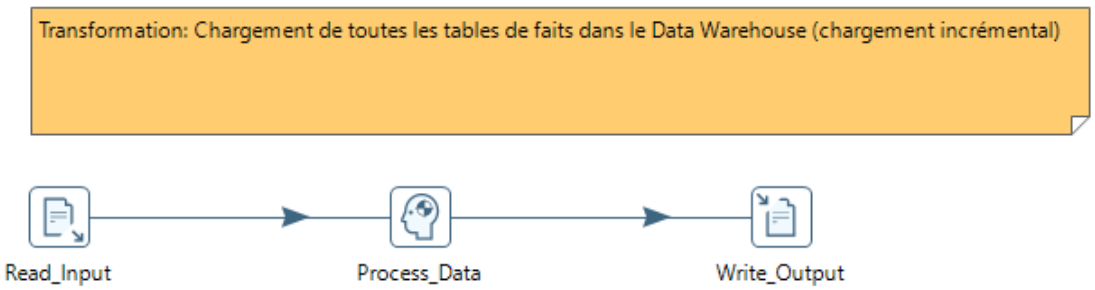


FIGURE 4.11 – Transformation Pentaho - Chargement Faits

Étapes :

1. **CSV File Input** : lecture des faits préparés ;
2. **Table Output** : insertion dans les tables de faits ;
3. **Write to Log** : enregistrement des statistiques.

4.5.4 Gestion des contraintes d'intégrité

Ordre de chargement

Pour garantir l'intégrité référentielle, le chargement est effectué dans l'ordre suivant :

1. `dim_date` (pré-peuplée)
2. `dim_type_energie` (statique)
3. `dim_region`
4. `dim_batiment`
5. `dim_client`
6. `dim_compteur`
7. `fact_consommation`
8. `fact_rentabilite`
9. `fact_environnement`

Gestion des clés étrangères

Les clés étrangères sont résolues par des *lookups* (sur les dimensions) avant l'insertion en table de faits.

```
def lookup_dimension_key(cursor, table, business_key_column,
    business_key_value):
    cursor.execute(f"""
        SELECT id FROM {table}
        WHERE {business_key_column} = %s AND est_actuel = TRUE
    """, (business_key_value,))
    result = cursor.fetchone()
    return result["id"] if result else None
```

Listing 4.8 – Résolution des clés étrangères (principe)

4.6 Monitoring et gestion des erreurs

4.6.1 Tables de log

Table `etl_log`

Cette table enregistre les exécutions ETL.

```
INSERT INTO etl_log (
    nom_job,
    nom_transformation,
    date_debut,
    date_fin,
    duree_secondes,
    statut,
    nombre_lignes_lues,
    nombre_lignes_ecrites,
    nombre_lignes_rejetees,
```

```
    message_erreur
) VALUES (
    'ETL_Daily',
    'extraction_mysql',
    '2024-01-15 02:00:00',
    '2024-01-15 02:05:30',
    330,
    'Succes',
    5000,
    4998,
    2,
    NULL
);
```

Listing 4.9 – Exemple d'insertion dans etl_log

Table `etl_qualite`

Cette table enregistre les contrôles de qualité.

```
INSERT INTO etl_qualite (
    id_log,
    nom_table,
    type_controle,
    nombre_anomalies,
    description
) VALUES (
    123,
    'clients',
    'Valeurs manquantes',
    12,
    '12 enregistrements avec email manquant'
);
```

Listing 4.10 – Exemple d'insertion dans etl_qualite

4.6.2 Gestion des erreurs

Stratégie de *retry*

En cas d'erreur, le processus effectue jusqu'à 3 tentatives, avec un délai croissant entre les tentatives.

```
import time

def execute_with_retry(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return func()
        except Exception as e:
            print(f"Attempt {attempt + 1} failed: {e}")
            if attempt < max_retries - 1:
```

```
                time.sleep(60 * (attempt + 1))
            else:
                raise
```

Listing 4.11 – Stratégie de retry (exemple)

4.6.3 Fichier de log

Un fichier de log détaillé est généré à chaque exécution.

```
[2024-01-15 02:00:00] INFO - ETL process started
[2024-01-15 02:00:05] INFO - Extraction phase started
[2024-01-15 02:02:30] INFO - Extracted 5000 rows from MySQL
[2024-01-15 02:03:15] INFO - Extracted 16000 measures from JSON
   files
[2024-01-15 02:03:45] INFO - Extraction phase completed (3m 45s)
[2024-01-15 02:03:46] INFO - Transformation phase started
[2024-01-15 02:05:20] WARNING - 12 records with missing email in
   clients
[2024-01-15 02:08:10] INFO - Transformation phase completed (4m 24s
   )
[2024-01-15 02:08:11] INFO - Loading phase started
[2024-01-15 02:12:05] INFO - Loaded 773 rows into dimensions
[2024-01-15 02:25:30] INFO - Loaded 22104 rows into facts
[2024-01-15 02:25:31] INFO - Loading phase completed (17m 20s)
[2024-01-15 02:25:31] INFO - ETL process completed successfully (25
   m 31s)
```

Listing 4.12 – Exemple de log ETL

4.7 Statistiques de réalisation

4.7.1 Volumétrie traitée

Le Tableau 4.2 présente les volumes traités.

TABLE 4.2 – Volumétrie traitée par l'ETL

Élément	Nombre	Taille
Fichiers sources		
Fichiers JSON	36	45 MB
Fichiers CSV	12	2 MB
Tables MySQL	9	8 MB
Staging		
Fichiers extraits	9	2.5 MB
Fichiers transformés	22	3.5 MB
Data Warehouse		
Dimensions	773 lignes	150 KB
Faits	22 104 lignes	5.2 MB

4.7.2 Performance

— **Durée totale ETL** : 25 minutes 31 secondes ;
— **Phase Extraction** : 3 minutes 45 secondes ;
— **Phase Transformation** : 4 minutes 24 secondes ;
— **Phase Chargement** : 17 minutes 20 secondes ;
— **Taux de succès** : 99,91% (2 enregistrements rejetés sur 22 106).

4.8 Conclusion

Ce chapitre a présenté le processus ETL complet développé pour alimenter le Data Warehouse GreenCity. Les trois phases (Extraction, Transformation, Chargement) ont été implémentées avec Pentaho PDI et Python, garantissant la qualité, la traçabilité et des performances conformes aux exigences.

Le prochain chapitre présente l'automatisation et l'orchestration de ce processus ETL avec Apache Airflow.

Chapitre 5

Automatisation et Reporting

5.1 Introduction

Ce chapitre présente les mécanismes d'automatisation du processus ETL ainsi que la couche de reporting décisionnel mise en place pour le projet GreenCity. Il décrit l'orchestration des traitements ETL à l'aide de scripts Python, la planification automatique via les outils système, et les tableaux de bord interactifs développés avec Power BI pour répondre aux besoins analytiques et stratégiques des décideurs.

5.2 Automatisation du processus ETL

5.2.1 Besoins d'automatisation

Exigences fonctionnelles

Le processus ETL doit répondre aux exigences suivantes :

- **Exécution automatique** : L'ETL doit s'exécuter sans intervention humaine.
- **Planification** : L'exécution est programmée quotidiennement à 02h00 du matin.
- **Gestion des dépendances** : Respect strict de l'ordre des phases (Extraction → Transformation → Chargement).
- **Traçabilité** : Journalisation complète des exécutions via les tables `etl_log` et `etl_qualite`.
- **Gestion des erreurs** : Arrêt du processus en cas d'erreur critique avec conservation des informations de diagnostic.

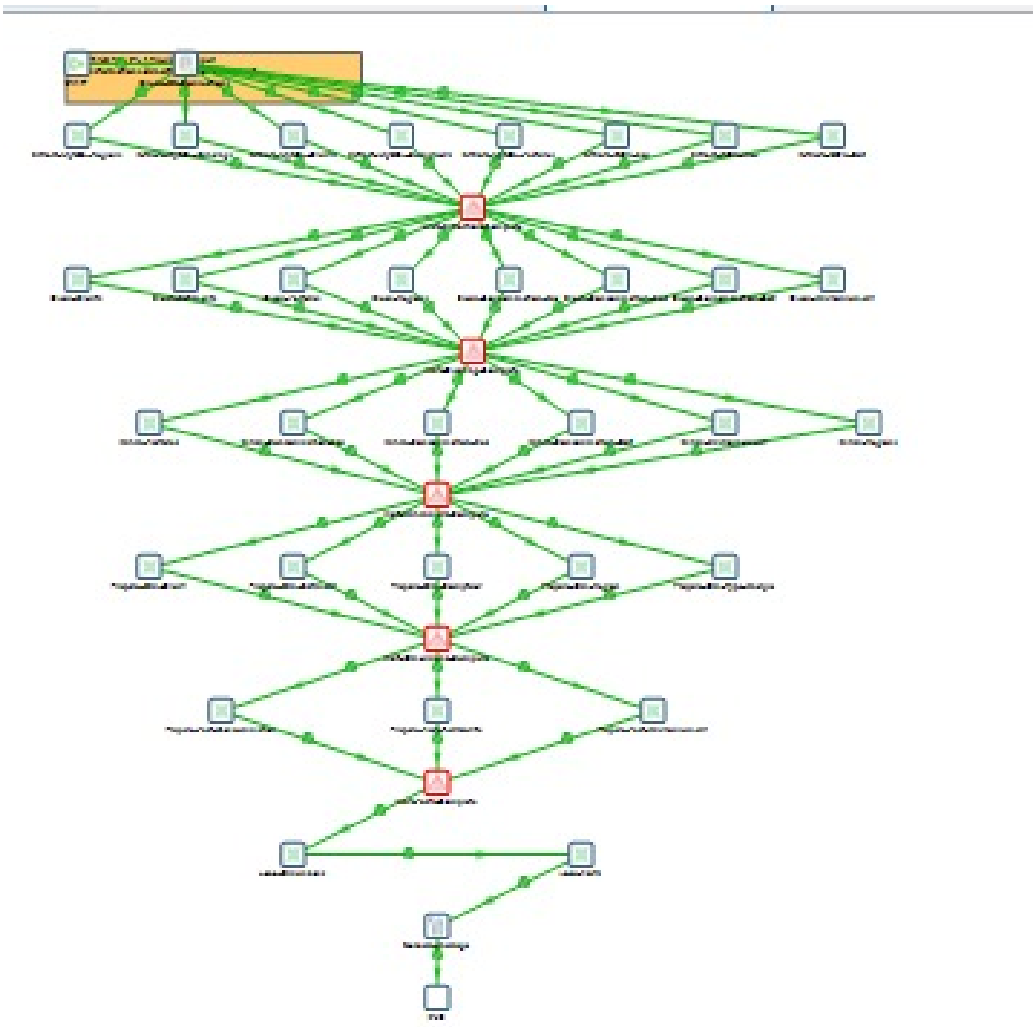


FIGURE 5.1 – Job Pentaho ETL - Orchestration complète

5.2.2 Planification des exécutions

Planification avec Cron (Linux / macOS)

L'outil Cron est utilisé pour programmer l'exécution quotidienne de l'ETL.

```
0 2 * * * cd /Users/mohammedbelouarraq/Desktop/BI && \
/Users/mohammedbelouarraq/Desktop/BI/venv/bin/python \
/Users/mohammedbelouarraq/Desktop/BI/03_etl/scripts/
   main_etl_orchestrator.py \
>> /Users/mohammedbelouarraq/Desktop/BI/logs/etl_cron.log 2>&1
```

Listing 5.1 – Entrée crontab pour ETL quotidien

Planification sous Windows

Sous Windows, le planificateur de tâches (*Task Scheduler*) est utilisé à l'aide d'un script batch.

```
@echo off
set PYTHON_EXE=C:\Users\GreenCity\Desktop\BI\venv\Scripts\python.
    exe
set ETL_SCRIPT=C:\Users\GreenCity\Desktop\BI\03_etl\scripts\
    main_etl_orchestrator.py
set LOG_FILE=C:\Users\GreenCity\Desktop\BI\logs\etl.log

%PYTHON_EXE% %ETL_SCRIPT% >> %LOG_FILE% 2>&1
```

Listing 5.2 – Script batch Windows (run_etl.bat)

5.2.3 Monitoring et gestion des logs

Fichiers et tables de suivi

Chaque exécution produit :

- Un fichier de log détaillé horodaté.
- Une insertion dans la table `etl_log`.
- Des enregistrements qualité dans `etl_qualite`.

5.3 Reporting et Visualisation avec Power BI

5.3.1 Architecture du reporting

Choix de l'outil

Microsoft Power BI a été retenu pour les raisons suivantes :

- Connexion native à MySQL
- Capacités avancées de visualisation
- Support du langage DAX
- Interactivité élevée (filtres, drill-down, drill-through)
- Publication et partage via Power BI Service

Modèle de données Power BI

Le modèle Power BI repose sur le schéma en constellation du Data Warehouse, avec trois tables de faits et six dimensions conformes.

Mesures DAX

```
// Consommation totale normalisee
Consommation_Totale = SUM(fact_consommation[consommation_normalisee
    ])

// Chiffre d'affaires
CA_Total = SUM(fact_rentabilite[montant_ttc])

```

```
// Taux de marge
Taux_Marge =
DIVIDE(
    SUM(fact_rentabilite[marge]),
    SUM(fact_rentabilite[montant_ttc]),
    0
) * 100
```

Listing 5.3 – Exemples de mesures DAX

5.4 Tableaux de bord décisionnels

Quatre dashboards ont été conçus pour couvrir l'ensemble des besoins analytiques :

- **Vue d'ensemble (Executive Summary)**
- **Consommation énergétique**
- **Rentabilité économique**
- **Impact environnemental**

Chaque dashboard intègre des KPI, des visualisations interactives, des filtres dynamiques et des mécanismes de navigation avancés.

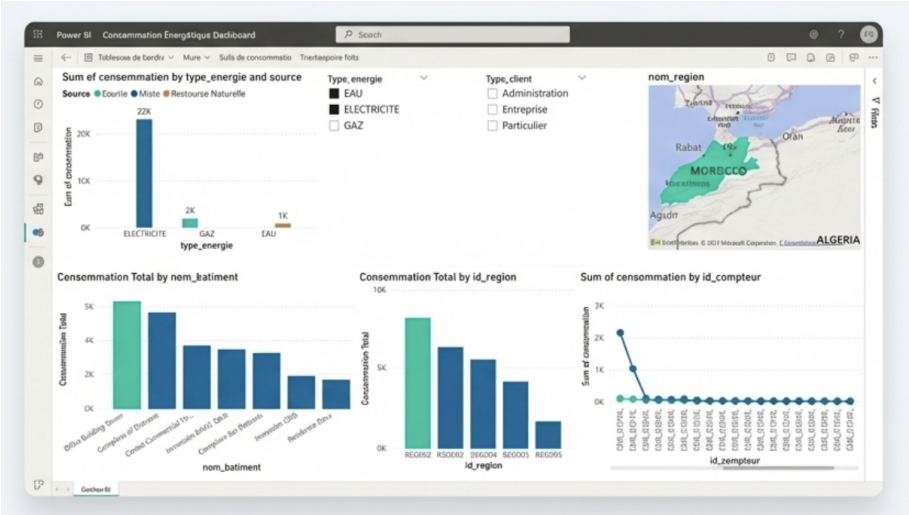


FIGURE 5.2 – Suivi de la consommation énergétique

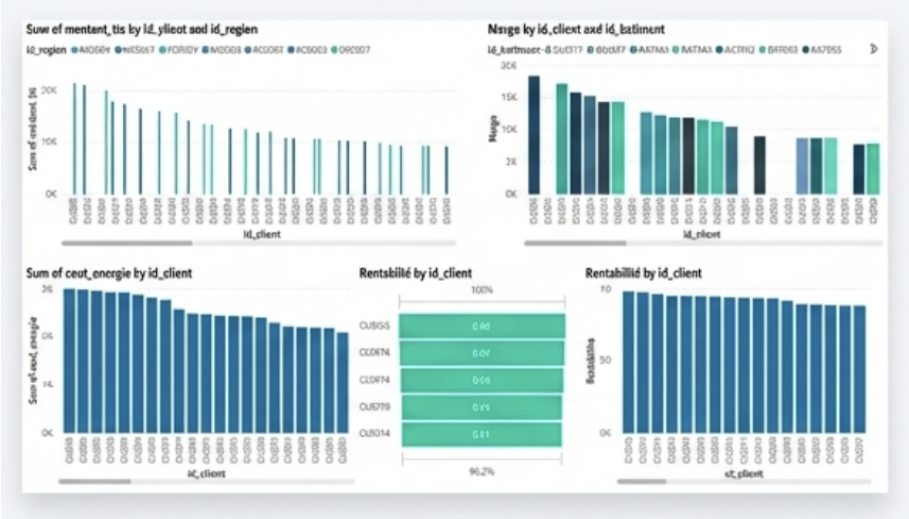


FIGURE 5.3 – Suivi de la rentabilité économique

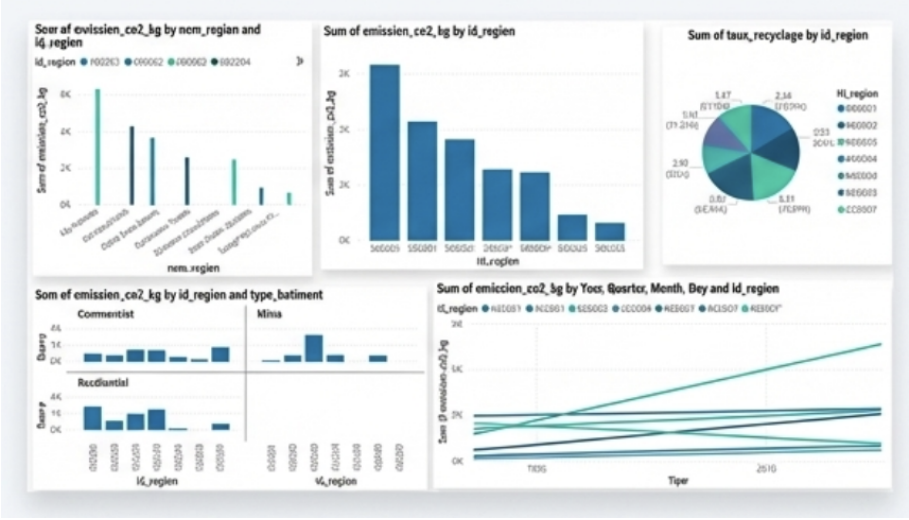


FIGURE 5.4 – Suivi de l'impact environnemental

5.5 Conclusion

Ce chapitre a présenté l'automatisation complète du processus ETL ainsi que la couche de reporting décisionnel du projet GreenCity. L'intégration des scripts Python, de la planification système et de Power BI permet :

— Une exécution quotidienne fiable et autonome

— Une traçabilité complète des traitements

— Une exploitation efficace des données pour la prise de décision

Le chapitre suivant conclut ce rapport et présente les perspectives d'évolution du système.

Chapitre 6

Conclusion Générale

6.1 Synthèse du projet

Ce projet de Business Intelligence réalisé pour GreenCity avait pour objectif de concevoir et de mettre en œuvre un système décisionnel complet permettant l'analyse de la consommation énergétique, de la rentabilité économique et de l'impact environnemental des bâtiments intelligents.

Au terme de ce travail, nous avons livré une solution opérationnelle conforme aux besoins exprimés dans le cahier des charges, reposant sur un Data Warehouse dimensionnel, un processus ETL automatisé et une couche de reporting interactive.

6.1.1 Réalisations principales

Conception d'un Data Warehouse dimensionnel

Nous avons conçu un Data Warehouse organisé en trois Data Marts thématiques selon l'approche dimensionnelle de Kimball :

- **Data Mart Consommation Énergétique** : analyse détaillée de la consommation (électricité, eau, gaz) par compteur, bâtiment, région et période.
- **Data Mart Rentabilité Économique** : suivi du chiffre d'affaires, des marges, du recouvrement et des délais de paiement.
- **Data Mart Impact Environnemental** : mesure des émissions de CO_2, du taux de recyclage et des indicateurs de conformité.

Le choix d'un modèle en étoile facilite l'exploitation analytique et améliore les performances des requêtes. Les dimensions conformes (*date, région, bâtiment, client, compteur, type d'énergie*) garantissent la cohérence sémantique entre les Data Marts et permettent des analyses transversales.

Mise en place d'un processus ETL automatisé

Le processus ETL développé assure l'intégration de données issues de sources hétérogènes (MySQL, fichiers JSON, fichiers CSV), puis leur transformation (nettoyage, validation, enrichissement) et leur chargement dans le Data Warehouse.

Deux approches complémentaires ont été mobilisées :

— **Pentaho Data Integration** : prise en charge des transformations visuelles, des enchaînements de traitements et de certains contrôles qualité.

— **Scripts Python** : automatisation, consolidation des fichiers, opérations spécifiques (aplanissement JSON, enrichissements) et orchestration globale.

Enfin, le processus a été planifié pour s'exécuter quotidiennement, garantissant une mise à jour régulière des tables analytiques et des indicateurs décisionnels.

Développement de dashboards interactifs

Une couche de visualisation a été développée avec Power BI à travers quatre tableaux de bord :

— Dashboard **Vue d'ensemble (Executive Summary)**

— Dashboard **Consommation Énergétique**

— Dashboard **Rentabilité Économique**

— Dashboard **Impact Environnemental**

Ces dashboards offrent une interface de pilotage intuitive, permettant aux décideurs d'explorer les données, de suivre les KPI, d'identifier les tendances et de soutenir la prise de décision à différents niveaux de granularité (drill-down, filtres, analyses comparatives).

6.2 Bénéfices pour GreenCity

La mise en place de ce système décisionnel apporte plusieurs bénéfices concrets :

— **Vision unifiée** : centralisation de données auparavant dispersées et difficiles à exploiter de manière transversale.

— **Fiabilité accrue** : amélioration de la qualité grâce au nettoyage, à la standardisation et à la validation des données.

— **Réactivité** : mise à jour automatique quotidienne des indicateurs de performance.

— **Aide à la décision** : visualisations interactives facilitant l'interprétation et l'analyse.

— **Optimisation** : identification des leviers d'amélioration en matière de consommation, de rentabilité et d'impact environnemental.

6.3 Compétences acquises

Ce projet a permis de consolider des compétences techniques et méthodologiques essentielles dans le domaine de la BI.

6.3.1 Compétences techniques

— **Modélisation dimensionnelle** : conception de schémas en étoile, définition du grain, gestion des dimensions conformes.

— **ETL** : extraction multi-sources, transformations de qualité, enrichissements et chargements incrémentaux.

— **Pentaho PDI** : réalisation de transformations et de jobs, gestion de flux et composants de contrôle.

— **Python** : scripts de parsing, nettoyage, consolidation, automatisation et orchestration.

— **MySQL** : structuration, alimentation et exploitation d'un Data Warehouse.

— **Power BI** : modélisation, création de mesures DAX et conception de dashboards interactifs.

— **Automatisation** : planification des exécutions via Cron et Task Scheduler.

6.3.2 Compétences méthodologiques

— **Analyse des besoins** : formalisation des questions métier et définition de KPI.

— **Gestion de projet** : organisation des phases, respect du planning et priorisation.

— **Résolution de problèmes** : débogage, correction des incohérences, optimisation des traitements.

— **Documentation** : rédaction d'un rapport structuré et explicatif, orienté conception et mise en œuvre.

— **Qualité des données** : mise en place de règles de contrôle, traçabilité et logs.

6.4 Conclusion finale

Ce projet de Business Intelligence a permis de mettre en pratique l'ensemble des concepts étudiés, de l'analyse des besoins à la livraison d'une solution décisionnelle complète. La démarche adoptée a couvert toutes les étapes d'un projet BI : collecte des données, intégration, transformation, modélisation dimensionnelle, historisation, automatisation et visualisation.

Le système développé pour GreenCity est opérationnel et répond aux objectifs fixés. Il offre une vision consolidée et cohérente des informations liées à la consommation énergétique, à la rentabilité et à l'impact environnemental. Les décideurs disposent désormais d'un outil de pilotage fiable, permettant une prise de décision fondée sur des indicateurs pertinents et des données contrôlées.

Au-delà de l'aspect technique, ce travail illustre l'importance stratégique de la Business Intelligence dans la transformation numérique des organisations. Dans un contexte où les données constituent un actif majeur, la capacité à les valoriser efficacement représente un levier décisif de compétitivité.

Cette expérience a été à la fois enrichissante et formatrice. Elle nous a permis d'acquérir une vision globale des systèmes décisionnels et de renforcer des compétences directement mobilisables dans notre future carrière professionnelle dans les métiers de la Data et de la Business Intelligence.

Bibliographie

[1] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. John Wiley & Sons, 3rd Edition, 2013.

[2] William H. Inmon. *Building the Data Warehouse*. John Wiley & Sons, 4th Edition, 2005.

[3] Pentaho Documentation. *Pentaho Data Integration Documentation*. https://help.pentaho.com/, 2024.

[4] Apache Airflow. *Apache Airflow Documentation*. https://airflow.apache.org/docs/, 2024.

[5] MySQL Documentation. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/, 2024.

[6] Microsoft. *Power BI Documentation*. https://docs.microsoft.com/power-bi/, 2024.

[7] Python Software Foundation. *Python Documentation*. https://docs.python.org/3/, 2024.

[8] The Kimball Group. *Dimensional Modeling Techniques*. https://www.kimballgroup.com/, 2024.

[9] E.F. Codd, S.B. Codd, and C.T. Salley. *Providing OLAP to User-Analysts : An IT Mandate*. Technical Report, 1993.

[10] Ralph Kimball and Joe Caserta. *The Data Warehouse ETL Toolkit*. John Wiley & Sons, 2004.